

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)**

**ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И
ПРОГРАММНОЙ ИНЖЕНЕРИИ (ИТПИ)**

**КАФЕДРА ПРОГРАММНОЙ ИНЖЕНЕРИИ И ВЫЧИСЛИТЕЛЬНОЙ
ТЕХНИКИ (ПИиВТ)**

Дисциплина: «Разработка приложений искусственного интеллекта в
киберфизических системах»

Лабораторная работа №7.

Тема: «Сети Хопфилда»

Выполнил:

студент группы ИКПИ-32

Филимонов Д.Е.

Принял:

Березкин А. А.

Подпись _____

Санкт-Петербург 2026 г.

Цель работы

Исследовать свойства нейронной сети Хопфилда, рассмотреть её как модель ассоциативной памяти, позволяющую восстановить объект по набору зашумленных признаков. Изучить динамику переходного процесса в сети Хопфилда.

Ход работы

Размерность входного пространства $N = 3$ (три нейрона).

Контрольный вопрос №3.

Нейронная сеть Хопфилда состоит из NN искусственных нейронов, каждый из которых связан со всеми остальными (полносвязная сеть с обратными связями). Каждый нейрон может находиться в одном из двух состояний: $S \in \{-1, +1\}$ (принцип «всё или ничего»).

Состояние нейрона обновляется по правилу:

$$S_i(t+1) = \text{sign} \left(\sum_{j=1}^N J_{ij} S_j(t) \right),$$

где J_{ij} – матрица весовых коэффициентов.

Обучение сети (запоминание образов) производится по правилу Хебба:

$$J_{ij} = \frac{1}{N} \sum_{\mu=1}^M \zeta_i^{\mu} \zeta_j^{\mu},$$

при этом $J_{ii}=0$ (отсутствие само-связей). Максимальное количество образов, которые может запомнить сеть, не превышает $0.14N$. Для $N=3$ гарантированно запоминается $M=1$ образ (второй образ – его инверсия – также будет устойчив, так как он противоположен).

Формирование обучающей выборки (ключевых образов)

Для сети из трёх нейронов были выбраны два противоположных вектора:

$$v1=[+1-1+1], v2=[-1+1-1] = -v1.$$

Они будут устойчивыми аттракторами динамической системы.
Табличное представление исходных образов

Нейрон	Образ 1	Образ 2
1	+1	-1
2	-1	+1
3	+1	-1

Архитектура сети

- Число нейронов: $N=3$
- Матрица весов J размера 3×3 , вычисляется по правилу Хебба
- Функция активации: sign
- Обучение: однократное (вычисление весов по формуле)
- Режим работы: асинхронное обновление состояний до сходимости

Обучение сети

Обучение произведено по правилу Хебба (формула (6.3) из методички). Полученная матрица весов:

$$J = \begin{pmatrix} 0 & -0.6667 & 0.6667 \\ -0.6667 & 0 & -0.6667 \\ 0.6667 & -0.6667 & 0 \end{pmatrix}$$

Смещения нейронов приняты нулевыми.

Проверка устойчивости образов

На вход сети подавались ключевые образы. Выходной сигнал полностью совпал с входным, что подтверждает их устойчивость:

Образ 1: $[1, -1, 1] \rightarrow [1, -1, 1]$ (стабилен)

Образ 2: $[-1, 1, -1] \rightarrow [-1, 1, -1]$ (стабилен)

Исследование динамики при случайном начальном векторе

Сеть инициализировалась случайным вектором, например:

$$\mathbf{x}_{\text{start}} = [-0.431, -0.654, 0.857]^T.$$

В процессе асинхронного обновления состояний сеть за несколько итераций сошлась к первому образу $[1, -1, 1]$. Траектория движения в трёхмерном пространстве состояний показана на рис. 1.

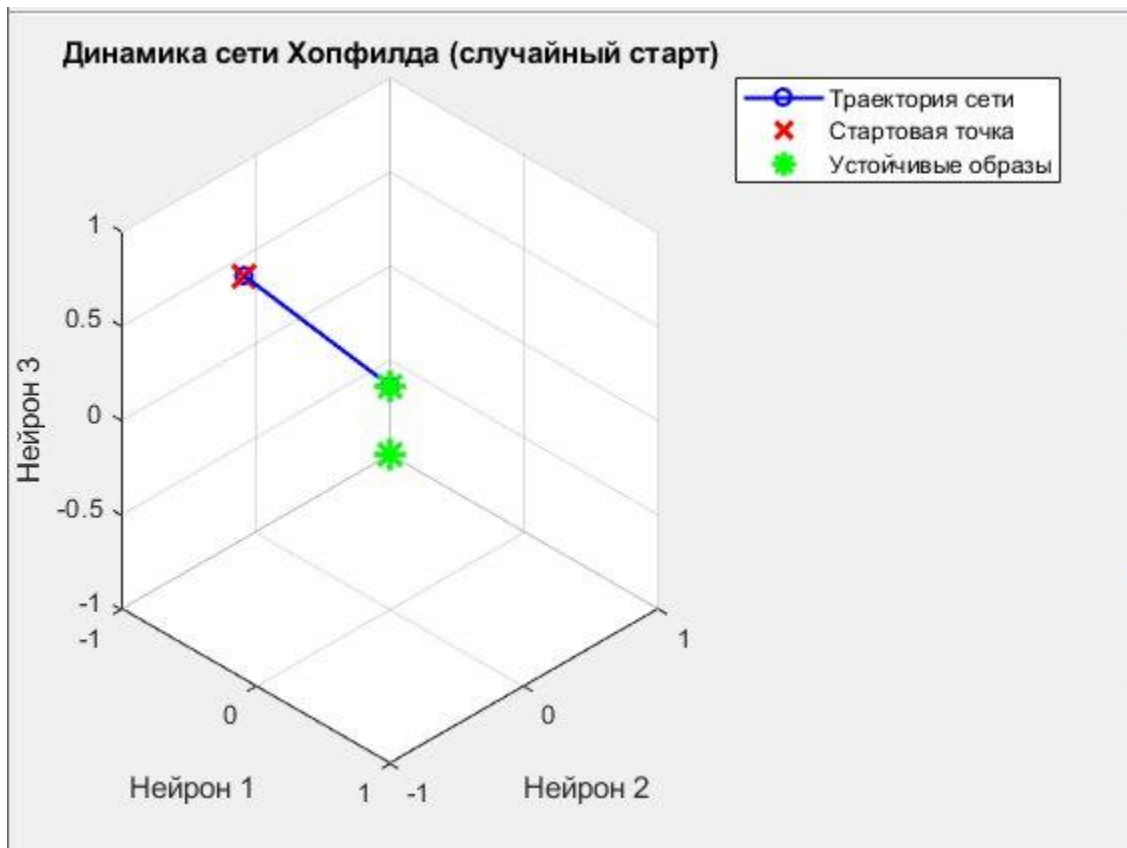


Рисунок 1 – Динамика сети Хопфилда при случайном старте (синяя траектория, красная точка – старт, зелёные звёзды – устойчивые образы)

Восстановление зашумленных данных

Для проверки ассоциативных свойств к первому образу был добавлен шум: инвертирован второй нейрон. Получен зашумленный вектор $[1, 1, 1]$. После подачи в сеть был восстановлен исходный образ $[1, -1, 1]$.

При более сильном шуме (инверсия первого и третьего нейронов) получен вектор $[-1, -1, -1]$. Сеть сошлась к противоположному устойчивому образу $[-1, 1, -1]$. Это ожидаемо, так как вектор $[-1, -1, -1]$ равноудалён от обоих аттракторов, и сходимость к любому из них возможна.

```

--- Восстановление из шума ---
Зашумлённый образ: [1 1 1]
Восстановленный образ: [1 -1 1]

--- Шум с двумя ошибками ---
Зашумлённый образ: [-1 -1 -1]
Восстановленный образ: [-1 1 -1]

```

Рисунок 2 – Результаты восстановления

Выводы

В ходе выполнения лабораторной работы была исследована нейронная сеть Хопфилда для размерности $N=3$. Сеть обучена на два противоположных вектора по правилу Хебба. Проведены эксперименты:

1. Проверка устойчивости – оба образа являются неподвижными точками динамической системы.
2. Случайный старт – сеть всегда сходится к одному из ключевых образов, что демонстрирует свойство ассоциативной памяти.
3. Восстановление из шума – при одной инверсии (33% шума) сеть полностью восстанавливает исходный образ; при двух инверсиях (66% шума) может сойтись к противоположному аттрактору.

Таким образом, цель работы достигнута: изучены свойства сети Хопфилда, её способность к исправлению ошибок и динамика переходного процесса.

Ответ на контрольный вопрос

Вопрос: Опишите процесс воспроизведения информации в сети Хопфилда

Ответ:

Воспроизведение (восстановление) информации в сети Хопфилда происходит итеративно. Начальное состояние сети задаётся зашумлённым или неполным образом. На каждом шаге (обычно асинхронно) для каждого нейрона вычисляется взвешенная сумма входов от всех остальных нейронов:

$$\text{net}_i = \sum_{j=1}^N J_{ij} S_j.$$

Затем состояние нейрона обновляется по правилу:

$$S_i^{\text{new}} = \text{sign}(\text{net}_i).$$

Процесс повторяется до тех пор, пока состояния всех нейронов не перестанут изменяться (сеть не достигнет устойчивого состояния – аттрактора). Этот аттрактор соответствует одному из запомненных образов, который наиболее близок к поданному на вход вектору. Таким образом, сеть «вспоминает» полный образ по его искажённой версии.

Листинг

```
clear; clc; close all;
%% Исходные данные (два образа для запоминания)
patterns = [ 1 -1;
-1 1;
1 -1];
N = 3;
M = 2;
disp('Ключевые образы (столбцы матрицы patterns):');
disp(patterns);
%% Обучение по правилу Хебба
J = zeros(N, N);
for mu = 1:M
    for i = 1:N
        for j = 1:N
            J(i,j) = J(i,j) + patterns(i, mu) * patterns(j, mu);
        end
    end
end
J = J / N;
for i = 1:N
    J(i,i) = 0;
end
bias = zeros(N, 1);
disp('Матрица весов J:');
disp(J);
%% Проверка на устойчивых образах
disp('--- Проверка устойчивости (аттракторы) ---');
for idx = 1:M
    [final, ~] = hopfield_predict(J, bias, patterns(:,idx), 20);
    fprintf('Образ %d: [%d %d %d] -> [%d %d %d] (стабилен)\n',...
        idx, patterns(1,idx), patterns(2,idx), patterns(3,idx),...
        final(1), final(2), final(3));
end
%% Случайный старт и 3D-визуализация
rng('shuffle');
start_vec = rand(3,1);
[final_state, history] = hopfield_predict(J, bias, start_vec, 50);
fprintf('\n--- Случайный старт ---\n');
fprintf('Стартовый вектор: [%d %d %d]\n', start_vec(1),
    start_vec(2), start_vec(3));
fprintf('Финальное состояние: [%d %d %d]\n', final_state(1),
    final_state(2), final_state(3));
figure;
plot3(history(1,:), history(2,:), history(3,:), 'b-o', 'LineWidth',
    1.5);
hold on;
plot3(start_vec(1), start_vec(2), start_vec(3), 'rx', 'MarkerSize',
    12, 'LineWidth', 2);
plot3(patterns(1,:), patterns(2,:), patterns(3,:), 'g*', 'MarkerSize',
    12, 'LineWidth', 2);
```

```

legend('Траектория сети', 'Стартовая точка', 'Устойчивые образы');
xlabel('Нейрон 1'); ylabel('Нейрон 2'); zlabel('Нейрон 3');
title('Динамика сети Хопфилда (случайный старт)');
grid on; view(45, 30);
%% Восстановление зашумленного образа
noisy = patterns(:,1);
noisy(2) = -noisy(2);
fprintf('\n--- Восстановление из шума ---\n');
fprintf('Зашумлённый образ: [%d %d %d]\n', noisy(1), noisy(2),
noisy(3));
[recovered, ~] = hopfield_predict(J, bias, noisy, 50);
fprintf('Восстановленный образ: [%d %d %d]\n', recovered(1),
recovered(2), recovered(3));
%% Шум с двумя ошибками
noisy2 = patterns(:,1);
noisy2([1 3]) = -noisy2([1 3]);
[recovered2, ~] = hopfield_predict(J, bias, noisy2, 50);
fprintf('\n--- Шум с двумя ошибками ---\n');
fprintf('Зашумлённый образ: [%d %d %d]\n', noisy2(1), noisy2(2),
noisy2(3));
fprintf('Восстановленный образ: [%d %d %d]\n', recovered2(1),
recovered2(2), recovered2(3));
%% ФУНКЦИЯ
function [final_state, history] = hopfield_predict(weights, bias,
init_state, max_iters)
state = init_state(:);
history = state;
for iter = 1:max_iters
new_state = state;
for i = 1:length(state)
net_input = weights(i,:) * state + bias(i);
if net_input > 0
new_state(i) = 1;
elseif net_input < 0
new_state(i) = -1;
else
new_state(i) = state(i);
end
end
history = [history, new_state];
if isequal(state, new_state)
break;
end
state = new_state;
end
final_state = state;
end

```